Lab 05 — Multi-stage builds and production-grade images

Theory — how to turn a 500 MB image that ships your compiler into a 200 MB image that ships only what runs; and what Buildah does where Docker uses BuildKit.

Objectives

- Understand why the image that **builds** your code must never be the image that **runs** it.
- Write a **multi-stage** build for Spring Boot and for Angular.
- Make an informed choice of base image (Debian/Ubuntu, Alpine, distroless).
- Know what BuildKit (Docker) and Buildah (Podman) offer: caching, secrets, stages.
- Connect image size to attack surface and deployment time.

---

1. The problem: the build tooling stays in the image

A first, naive Dockerfile for a Java API:

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn package -DskipTests
ENTRYPOINT ["java","-jar","/app/target/api.jar"]
```

It works. It also produces an image of **800 MB to 1 GB** that carries a full JDK (compiler, debugging tools), Maven, the local `~/.m2` repository with hundreds of JARs, your **source code**, your tests, and the final JAR. Production needs only the JRE and the JAR: about 200 MB.

The damage goes beyond cosmetics:

- **Security.** Anyone who gets the image gets your source code. Every bundled tool (compiler, `curl`, `git`, shell) is one more thing an attacker can turn against you, and one more line in the vulnerability scan report.
- **Cost.** Every deployment moves 800 MB to every node; every registry stores it, plus the retained older versions.
- **Time.** If a node does not have the image yet, the download is part of container startup. During a production incident at 3 a.m., that difference hurts.

> → SECURITY
> An image's **attack surface** is everything an attacker can *use* once they achieve code execution: a shell to explore with, `curl` to exfiltrate data, a compiler to craft tools, a package manager to install more. Every binary you leave out forces one extra step on them. That is why scanners (Trivy, Grype) count packages, and why "minimal" is about more than megabytes.

2. Multi-stage

A Dockerfile can contain **several** `FROM` **instructions**. Each one opens a *stage* — an independent build environment. Only the **last** stage becomes the final image; the others are thrown away.

`COPY --from=<stage>` pulls files out of an earlier stage.

```
# ---------- stage 1: build ----------
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# ---------- stage 2: runtime ----------
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/api.jar app.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

The final image contains **a JRE and a JAR** — no Maven, no JDK, no sources, no tests, no `.m2`. Nothing that happened in the `build` stage leaves any trace, not even in hidden layers: those layers are simply not part of the image.

REMEMBER
Multi-stage is also the only truly reliable protection for build secrets: whatever lands only in a discarded stage does not exist in the final image. One caveat: `COPY --from=build /app /app` would copy everything back, secrets included. Copy only the artefact.

The same pattern for Angular:

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build                      # produces dist/

FROM docker.io/library/nginx:alpine
COPY --from=build /app/dist/my-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

→ ANGULAR
`ng build` compiles the TypeScript components, bundles everything into a handful of `.js` and `.css` files plus an `index.html`, minifies them, and fingerprints their names (`main-a1b2c3.js`) for browser caching. The output is **static**: any file server can serve it. `ng serve`, by contrast, is a development server that recompiles on every change — invaluable on your workstation, useless in production.

One point matters above all: **Node does not survive** the build. An Angular front end in production is nothing but HTML, CSS and JavaScript; it needs no server-side JavaScript engine. You **never** containerise `ng serve`.

3. Choosing a base image

Base	Size	Advantages	Drawbacks
`debian` / `ubuntu`	75-120 MB	Everything works, full tooling, `glibc`	Heavy; many packages, so many CVEs
`*-slim`	30-80 MB	Good compromise, still Debian	Fewer tools installed
`alpine`	5-10 MB	Very light, efficient `apk`	Uses **musl** rather than `glibc`
distroless	20-50 MB	No shell, no package manager	Hard to debug, no `exec sh`

> **→ LINUX**
> The **C library** (`libc`) sits between programs and the kernel: `printf` , `malloc` , DNS resolution, locales. Almost every Linux binary depends on it. `glibc` (GNU) is the historical implementation, rich and compatible; `musl` is a minimalist rewrite that Alpine picked for its small size. A binary compiled against one will not load with the other; `ldd --version` inside the container tells you which one you have.

The Alpine trap deserves a closer look. Most programs run fine on `musl` , but not all of them: native binaries compiled for `glibc` refuse to start, some native Java libraries (compression, cryptography, PDF generation) fail with `UnsatisfiedLinkError` , and subtle differences show up in DNS resolution and locales. Some Java workloads have also measured slower memory allocation.

In practice, for Spring Boot, `eclipse-temurin:21-jre-alpine` covers the vast majority of cases and halves the image size. If a native dependency breaks, fall back to `eclipse-temurin:21-jre` (Ubuntu). Test the choice — never settle it on principle.

Distroless images (from Google) contain only the runtime and your application: no shell, no `ls` , no package manager. The attack surface is minimal, but `podman exec -it container sh` no longer works — you need to have planned your observability some other way.

4. What really weighs

Four levers, from most to least effective:

1. **Multi-stage** — removes the build tooling. This is the big one: 800 MB → 200 MB.
2. **The base image** — `-jre` instead of `-jdk` , `alpine` instead of `ubuntu` .
3. `.dockerignore` — keeps `.git` , `node_modules` and `target` out of the image.
4. **Combining install and clean-up** in a single `RUN` .

One thing has **no** effect at all: deleting files in a later layer. They stay in the image (lab 02). And the layer count alone barely moves the size.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman history my-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head
```

5. BuildKit and Buildah

Docker builds with **BuildKit**; Podman builds with **Buildah**. Both read the same Dockerfile and offer the same useful features:

- **Unused stages are never built.** A stage that contributes nothing to the final image is skipped — Buildah prints `[2/3]`, `[3/3]` and `[1/3]` never appears.
- `--target` stops the build at a chosen stage: `podman build --target build -t api-build .` gives you the compilation stage as an image you can inspect.
- **Persistent caches.** `RUN --mount=type=cache,target=/root/.m2 mvn package` keeps the Maven repository **across builds** without putting it in the image. On a CI agent the speedup is dramatic.
- **Secrets.** `RUN --mount=type=secret,id=npmrc …` exposes a file for the duration of a single instruction, without ever writing it to a layer (lab 08).

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests
```

PODMAN
One visible difference: BuildKit builds independent stages **in parallel**, Buildah builds them one at a time. Another: Buildah simply **ignores** the `# syntax=docker/dockerfile:1` line that enables extended syntax in Docker — the `--mount` options work without it. And the `type=cache` data lives in your user storage (`~/.local/share/containers/storage`), not in a daemon: two users on the same CI server each have their own.

6. Spring Boot: the layers of the JAR

A Spring Boot JAR weighs 50 MB: 45 MB of dependencies that almost never change, and 5 MB of code that changes with every commit. Copied as one block, it becomes a single 50 MB layer that every deployment transfers in full. Spring Boot can split it for you:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
COPY target/api.jar api.jar
RUN java -Djarmode=tools -jar api.jar extract --layers --destination extracted

FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=extract /app/extracted/dependencies/ ./
COPY --from=extract /app/extracted/spring-boot-loader/ ./
COPY --from=extract /app/extracted/snapshot-dependencies/ ./
COPY --from=extract /app/extracted/application/ ./
ENTRYPOINT ["java","-jar","app.jar"]
```

The dependencies become a stable layer, the code a small volatile one: a deployment now transfers just a few MB. What matters is the **principle** — lab 04's layer-ordering rule, applied inside a JAR.

7. In the workplace

- **One Dockerfile per service**, multi-stage, versioned alongside the code. CI needs neither Maven nor Node: `podman build` (or `docker build`) is enough, so the CI build and the workstation build are guaranteed identical.
- **Tests** often run in a dedicated stage (`RUN mvn test`), so a red test fails the image build.
- **Vulnerability scanning** (Trivy, Grype) targets the final image. A minimal image yields a short report that someone actually reads — a 1 GB image yields 300 CVEs that nobody will.
- **The final image runs as non-root**, on a port above 1024, without a shell where possible.

Remember

- The image that builds your code should never be the image that runs it — that is the whole point of multi-stage.
- Several `FROM`s mean several stages; only the last becomes the image, and `COPY --from` brings the artefact into it.
- Node has no place in the final image of an Angular front end: nginx serves the static files.
- Prefer `-jre` over `-jdk`, `alpine` when native dependencies allow it, and distroless when you can live without a shell.
- Alpine ships `musl`, not `glibc`: validate with a test, never on principle.
- BuildKit and Buildah both offer `--target`, persistent caches and build secrets; Buildah ignores `# syntax=` and does not parallelise.
- Deleting a file in a later layer does not shrink the image.

Vocabulary

stage: a build step opened by a `FROM`. — **`COPY --from`**: fetches files from another stage or image. — **`--target`**: stops the build at a given stage. — **distroless**: an image with no shell and no package manager. — **musl / glibc**: two implementations of the C library. — **BuildKit / Buildah**: Docker's and Podman's build engines. — **cache mount**: a cache that persists across builds, outside the image. — **attack surface**: the exploitable components present in an image.